The Kalman Gain, Verified

A ground-up guide to a molecular switching model,
what it should satisfy, and how Lean proves it

Jeffrey Lee · Brown CS 1715, Formal Proof and Verification · November 2026

At a glance

- **What.** A simple model of how a molecule moves (a *switching diffusion*), a filter that tracks it from noisy measurements (the *Kalman filter*), and 17 facts about both, proved in the proof assistant Lean 4 with its mathematics library Mathlib.
- **Who it is for.** Anyone who knows calculus, a little linear algebra and a little probability. No physics or chemistry is assumed.
- **How to read it.** Part I builds the model from scratch. Part II says what is worth proving about it and what the proofs assume. Part III shows how the model is defined in Lean and how the theorems are stated. Appendix A sketches every proof. Each section ends with a one-line *intuition*.

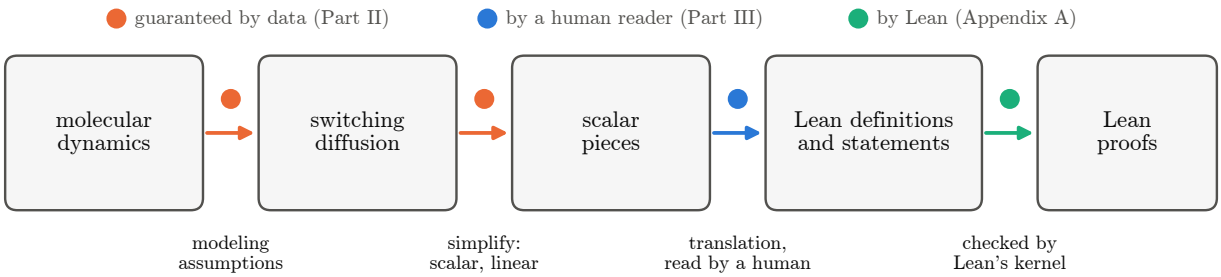


Figure 1. The road from a real molecule to a checked theorem. The first two steps are modeling choices that only data can support (Part II); the third is a translation that a person must read (Part III); the last is checked by Lean (Appendix A).

Contents

Part I The model, from the ground up **3**
1 A molecule is a point that moves . . . 3
2 The energy landscape . . . 3
3 How the molecule moves: downhill plus random kicks . . . 4
4 Equilibrium: where the molecule spends its time . . . 5
5 The surrogate: a switching diffusion . . . 6
6 Inside one basin: the Ornstein–Uhlenbeck process . . . 8
7 Sensors: what we actually see . . . 9
8 The Kalman filter: predict, then correct . . . 9
9 Innovations: the filter's surprises . . . 12
10 Two ways the model can be wrong, and their fingerprints . . . 12

Part II What should be proved, and what is assumed **15**
11 The verification agenda . . . 15
12 Assumptions: what the theorems take as given . . . 15
13 What the theorems give an MD run . . . 16

Part III The model in Lean **18**
14 Lean in five minutes . . . 18
15 The file and its trust boundary . . . 19
16 The definitions . . . 19
17 The seventeen verified statements . . . 21
18 The verification record . . . 25

Appendices **25**
A How each result is proved . . . 26
B Mathlib facts used . . . 29
C Symbols . . . 30
D Reproducing everything . . . 30

PART I

The model, from the ground up

1 A molecule is a point that moves

A molecule is a handful of atoms, from a few to many thousands. Each atom sits at a point in space, given by three numbers, so a molecule with N atoms is described by $3N$ numbers. Listing them gives one point

$$x = (x_1, y_1, z_1, \ldots, x_N, y_N, z_N) \in \mathbb{R}^{3N},$$

called the *configuration*. As the atoms jiggle, x moves. A *molecular dynamics* (MD) simulation is a program that computes this motion and saves a snapshot of x every Δt; each snapshot is a *frame*.

Nobody looks at all $3N$ numbers. We look at *coordinates*: real-valued functions of x that describe the shape, such as the distance between two atoms, or the *torsion* φ, the twist about the bond between atoms 2 and 3 (Figure 2). Precisely, φ is the angle between the plane through atoms 1, 2, 3 and the plane through atoms 2, 3, 4. A *coordinate family* is a group of similar coordinates, for example all the backbone torsions of a protein.

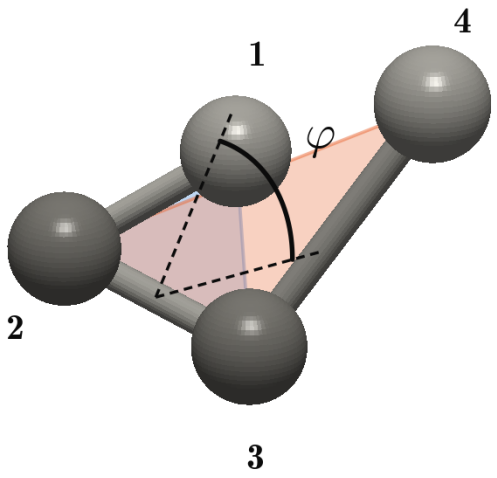


Figure 2. Four atoms joined by three bonds. The torsion φ is the angle between the blue triangle (atoms 1, 2, 3) and the orange one (atoms 2, 3, 4), measured about the bond 2–3; here $\varphi = 62°$. The configuration of these four atoms is one point of $\mathbb{R}^{12}$, and φ is a function of that point.

> **Intuition:** *a molecule is a moving point in a space of many dimensions; a coordinate is a function of that point.*

2 The energy landscape

Every configuration x has an *energy* $V(x)$, a real number computed from the atom positions. One fact about it is enough here: low energy means a comfortable shape. So think of $V\colon \mathbb{R}^{3N} \to \mathbb{R}$ as a landscape. Its valleys are the stable shapes, called *basins*; the mountain passes between valleys are the routes from one shape to another.

Definition (basin)

Let $c_1, \ldots, c_n$ be the local minima of V. The *basin* of c_i is the set of starting points from which steepest descent, $\dot{x} = -\nabla V(x)$, ends at c_i.

Since $\mathbb{R}^{3N}$ cannot be drawn, Figure 3 shows a toy landscape over a plane, with three basins called A, B and C. The same toy is used throughout Part I.

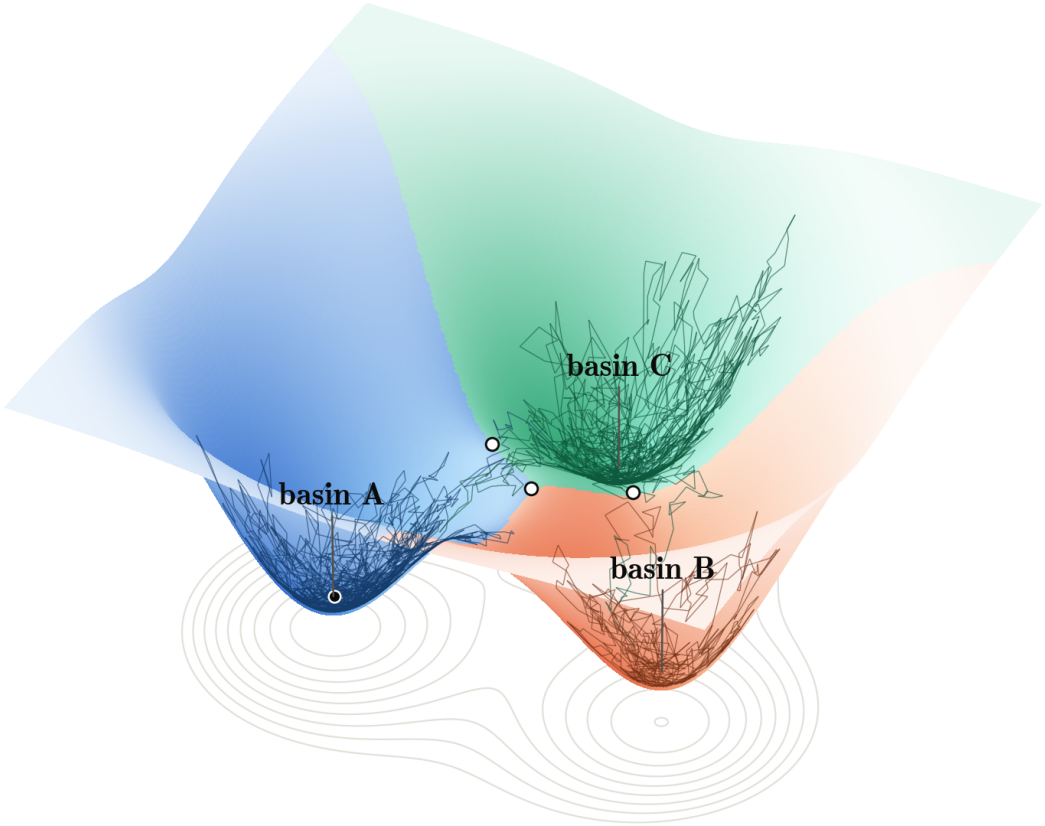


Figure 3. A toy energy landscape $V(x_1, x_2)$ with three basins (blue A, orange B, green C); white dots mark the passes. The dark path is a simulated motion (Section 3): it rattles inside a basin and now and then crosses a pass. The passes lie 0.9 to 1.7 energy units above the valley floors.

Intuition: *valleys are shapes the molecule likes; passes are the ways between them.*

3 How the molecule moves: downhill plus random kicks

A standard simplified model of MD motion is gradient descent with noise:

$$x_{k+1} = x_k - \nabla V(x_k)\,\Delta t + \sqrt{2\theta\,\Delta t}\;\xi_k, \qquad \xi_k \sim \mathcal{N}(0, I) \text{ independent.} \tag{1}$$

The first term pulls x downhill; the second adds a random kick whose size is set by $\theta > 0$ (physicists call θ the temperature; the toy uses $\theta = 0.5$). This is the overdamped Langevin equation, taken in small time steps.

Two time scales appear (Figure 4a).

- **Fast:** inside a basin, pull and kicks balance, and x rattles about the bottom. In the toy, the rattle forgets itself in about 0.25 time units.
- **Slow:** now and then a lucky run of kicks carries x over a pass, and it starts rattling in another basin. Hops are rare because they need many kicks in the same direction; in the toy, a stay in one basin lasts 8.6 time units on average.

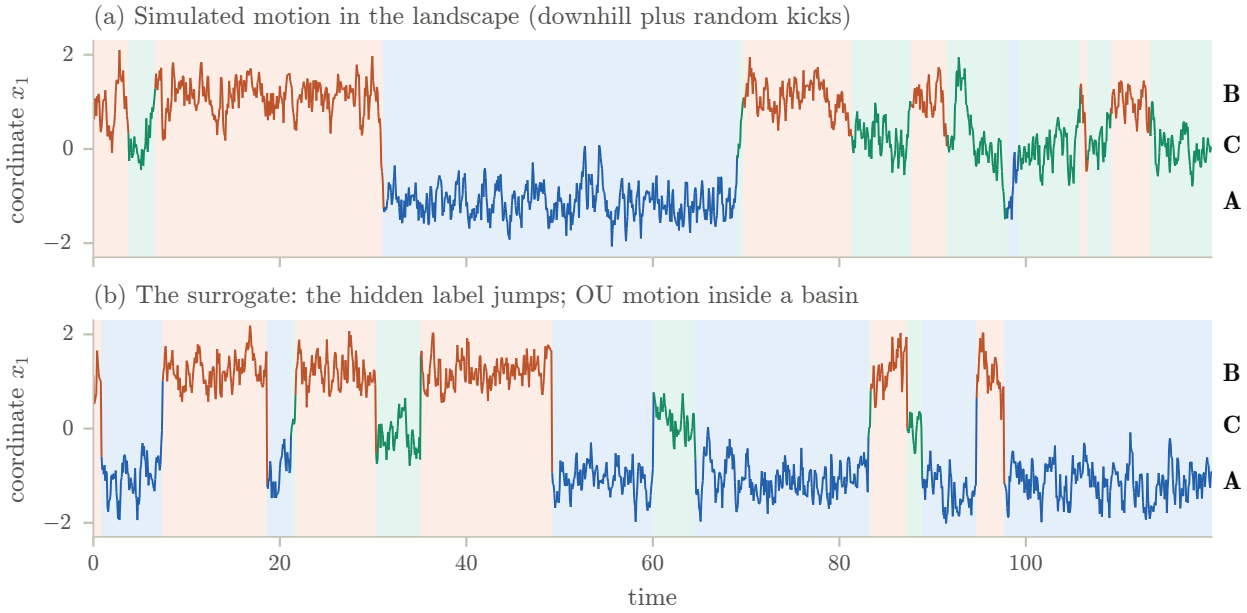


Figure 4. (a) The coordinate x_1 of one simulated run of (1), colored by basin: fast rattling, rare hops. (b) The surrogate model of Section 5, with its parameters read off the simulated runs (40 runs of 6,250 frames, 2,327 hops in all). It reproduces both time scales with far simpler mechanics.

> **Intuition:** *a fast rattle inside a basin and rare hops between basins: two very different clocks.*

4 Equilibrium: where the molecule spends its time

Run (1) for a long time and record where x is. For a long run and a small Δt, the fraction of time spent in a region A is close to $\int_A \pi(x)\,dx$, where

$$\pi(x) = \frac{e^{-V(x)/\theta}}{Z}, \qquad Z = \int e^{-V(x)/\theta}\,dx. \tag{2}$$

This is the *Boltzmann distribution*; we take it as a known fact about (1). Deep valleys get exponentially more time. The *population* of basin i is $p_i = \int_{\text{basin } i} \pi(x)\,dx$, its long-run share of time (Figure 5). In the toy, $p_A = 0.49$, $p_B = 0.30$ and $p_C = 0.21$.

The equilibrium is also *reversible*: a long equilibrium movie looks statistically the same played backwards. In particular, hops from i to j happen as often as hops from j to i. In the simulated runs the measured flows of each pair agree to within 6%, which is sampling noise. A simplified model should keep both features, the populations and the balanced flows. That is result M of Part III.

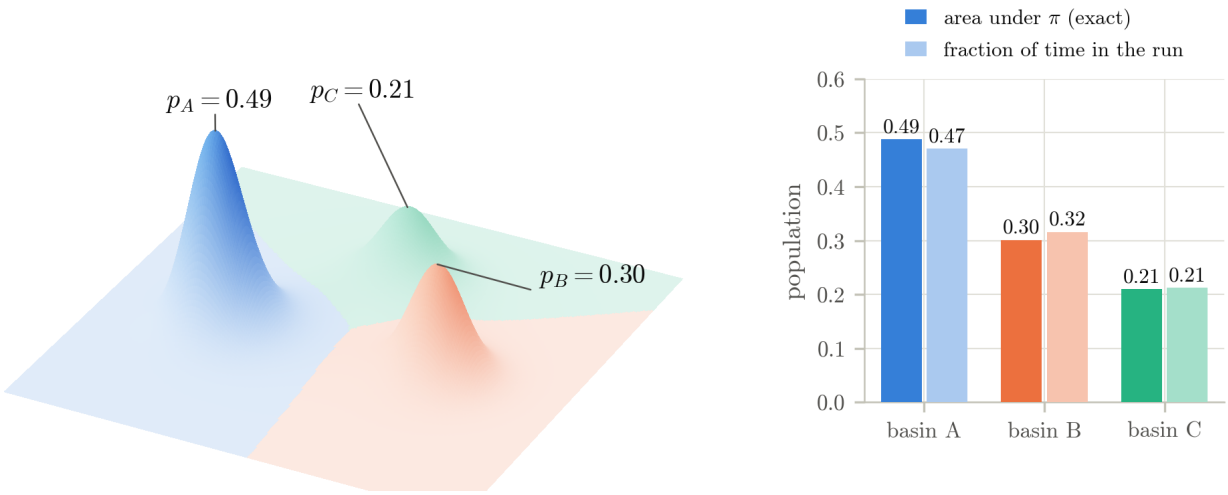


Figure 5. Left: the equilibrium density π of (2) for the toy landscape, colored by basin, with each basin's population. Right: the populations (exact, from π) against the fraction of time the simulated runs spent in each basin.

> **Intuition:** *in the long run, the time spent near a shape is proportional to* $e^{-energy/\theta}$.

5 The surrogate: a switching diffusion

MD is expensive and lives in thousands of dimensions. The research model replaces it by two simple pieces (Figure 6).

1. A hidden *basin label* $s_t \in \{1, \dots, n\}$ that jumps between basins at random times. From basin i it jumps to basin j at *rate* k_{ij}: in a short time dt the jump happens with probability about $k_{ij}\, dt$.

2. Inside the current basin, the slow coordinates follow an *Ornstein–Uhlenbeck* (OU) process: pulled back toward the basin's center and kicked at random (Section 6).

A diffusion whose rules switch with a hidden label is a *switching diffusion*. It keeps the two clocks of Section 3: OU rattle inside a basin, jumps between basins.

The jump rule. With the slow coordinates held fixed, the research model uses

$$k_{ij} = \kappa_{ij}\sqrt{p_j/p_i}, \qquad \kappa_{ij} = \kappa_{ji} > 0. \tag{3}$$

Here κ_{ij} is a symmetric attempt rate for the pair $\{i, j\}$. The square root speeds up jumps toward the more populated basin and slows the reverse by the same factor. Writing $p_i \propto e^{-V_i/\theta}$ gives $k_{ij} = \kappa_{ij}\, e^{-(V_j - V_i)/2\theta}$: the energy difference is split evenly between the two directions. (Rates of this form are known in the MD literature as the square-root approximation.) In the full model p_i and κ_{ij} may depend on the slow coordinates; result M is about one fixed value of them.

The generator. Collect the rates in a matrix L with $L_{ij} = k_{ij}$ for $i \neq j$ and $L_{ii} = -\sum_{j \neq i} k_{ij}$, so that every row sums to 0. If $q(t)$ is the row vector of probabilities of the label at time t, then $\frac{d}{dt}\, q(t) = q(t)\, L$. A population vector p is *stationary* when $p^{\mathsf{T}} L = 0$: started at p, the probabilities never change. Figure 7 shows a three-basin example.

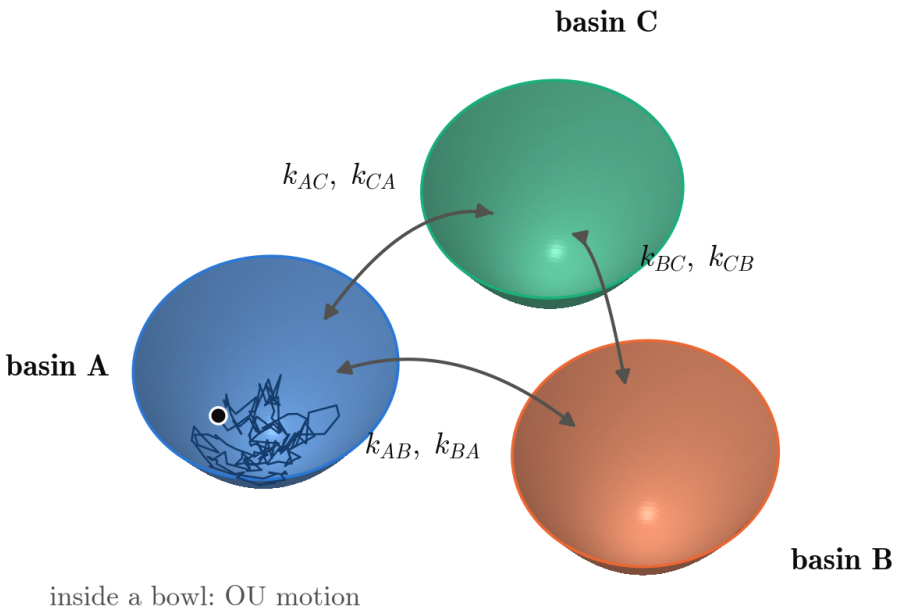


Figure 6. The switching diffusion. Each basin becomes a quadratic bowl in which the slow coordinates follow OU motion (dark path and ball); the hidden label jumps between bowls at rates k_{ij}. Fitted to the toy runs, the bowls have equilibrium variances 0.12, 0.12, 0.13 in x_1 and per-frame decays 0.72, 0.72, 0.75.

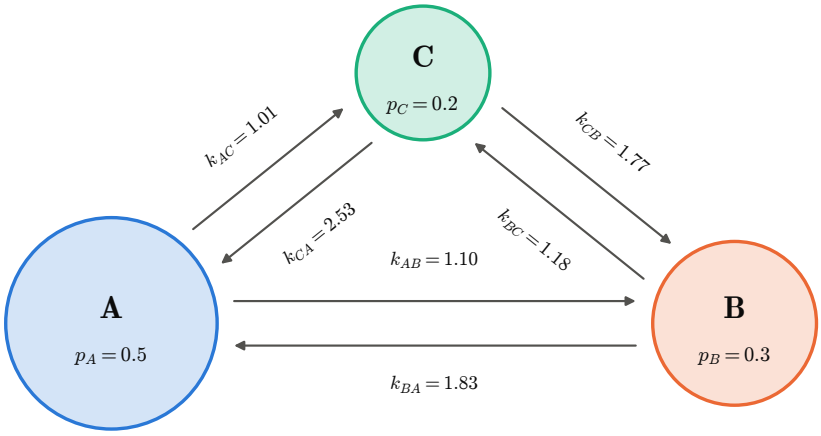


Figure 7. The jump rule (3) for populations $p = (0.5, 0.3, 0.2)$ and random symmetric prefactors ($\kappa_{AB} = 1.416$, $\kappa_{AC} = 1.599$, $\kappa_{BC} = 1.449$). Each pair trades equal flows both ways: $p_A k_{AB} = p_B k_{BA} = 0.548$, $p_A k_{AC} = p_C k_{CA} = 0.506$ and $p_B k_{BC} = p_C k_{CB} = 0.355$. In floating point the largest entry of $|p^{\mathsf{T}} L|$ is 6×10^{-17}, and 200,000 simulated jumps occupy the basins in proportions 0.501, 0.300, 0.199.

Intuition: *replace the landscape by bowls, and the passes by a weighted coin flip.*

6 Inside one basin: the Ornstein–Uhlenbeck process

Near the bottom c of a valley, V is close to a parabola. In one coordinate, $V(x) \approx V(c) + \frac{k}{2}(x-c)^2$, so the pull $-V'(x) = -k(x-c)$ is linear in the distance from the bottom. A process pulled back linearly and kicked at random is an OU process. Sampled every Δt and shifted so that the center is 0, it obeys exactly

$$z_{n+1} = a\, z_n + w_n, \qquad w_n \sim \mathcal{N}(0, Q), \qquad a = e^{-\Delta t/\tau_c}, \quad Q = \sigma^2(1-a^2), \tag{4}$$

with independent w_n. Here τ_c is the *relaxation time* and σ^2 the *equilibrium variance*. For (1), $\tau_c = 1/k$ and $\sigma^2 = \theta/k$: a flat basin (small k) is wide and slow.

Two consequences follow by induction on n:

$$\mathbb{E}[z_n] = a^n\, \mathbb{E}[z_0], \qquad \mathrm{Var}(z_n) = a^{2n}\, \mathrm{Var}(z_0) + \sigma^2(1-a^{2n}).$$

The mean forgets the start by a factor a per frame, and the variance tends to σ^2 (Figure 8). The rule that pushes a variance P through one step,

$$P \;\mapsto\; a^2 P + \sigma^2(1-a^2),$$

is the Lean definition `ouPredict`. It leaves σ^2 unchanged: if $\mathrm{Var}(z_n) = \sigma^2$ then $\mathrm{Var}(z_{n+1}) = \sigma^2$.

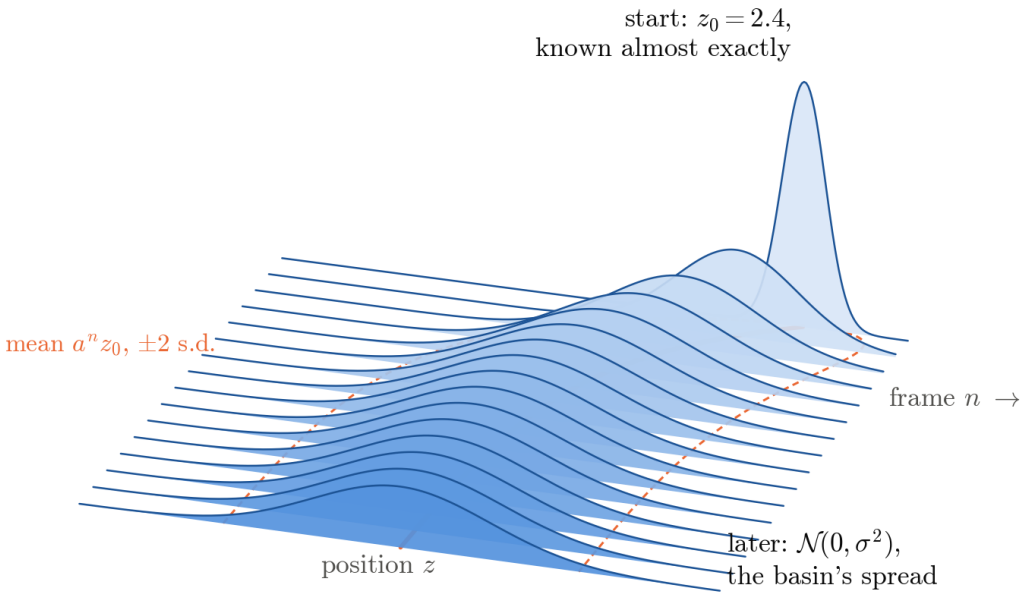


Figure 8. An OU mode forgets its start. Each slice is the density of z_n for a start $z_0 = 2.4$ known almost exactly, with relaxation time $\tau_c = 10$ frames (so $a = 0.905$) and $\sigma^2 = 1$. The orange lines on the floor are the mean $a^n z_0$ and two standard deviations either side.

Intuition: *inside a basin, uncertainty grows to the basin's width and stops there.*

7 Sensors: what we actually see

We never see the slow state directly. Each coordinate family c acts as a noisy *sensor* of it:

$$y_c = z + v_c, \qquad v_c \sim \mathcal{N}(0, R_c),$$

with v_c independent of z, of the other sensors and of the past. The number R_c is the sensor's *noise variance*. (In general a family measures a linear function $H_c z$ of a vector of slow coordinates; the formalization treats the scalar case $H_c = 1$.)

> **Intuition:** *a slow hidden state, seen through several noisy windows.*

8 The Kalman filter: predict, then correct

The filter keeps a best guess m of the hidden state and the variance P of its error. Figure 9 shows what happens in one frame.

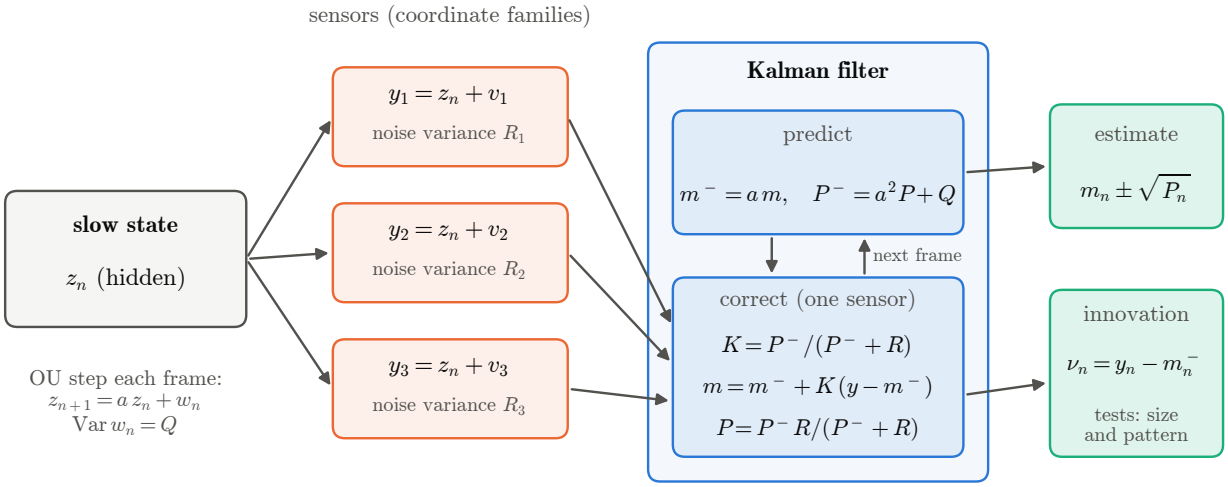


Figure 9. One frame. The hidden state takes an OU step; each sensor reports it with its own noise; the filter predicts, then corrects with each reading; it outputs an estimate with its uncertainty, and an innovation for each reading (Section 9).

Correct: the navigator and the lookout

Think of a ship's navigator, whose prediction m^- of the position misses the truth by $e = z - m^-$ with $\mathrm{Var}(e) = P$, and a lookout, whose sighting $y = z + v$ misses by v with $\mathrm{Var}(v) = R$, independent of e. Blend the two with a *gain* $K \in \mathbb{R}$:

$$m = m^- + K\,(y - m^-).$$

$K = 0$ ignores the sighting and $K = 1$ ignores the prediction. The new error is $z - m = (1 - K)\,e - K\,v$. Two rules give its variance: $\mathrm{Var}(cX) = c^2\,\mathrm{Var}(X)$, and $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ when X and Y are independent. Hence

$$\mathrm{Var}(z - m) = (1 - K)^2 P + K^2 R \;=\; \mathrm{postVar}(P, R, K). \tag{5}$$

Which gain is best? Completing the square in K (using $P + R > 0$):

$$\mathrm{postVar}(P, R, K) = (P + R)\,(K - K^*)^2 + \frac{PR}{P + R}, \qquad K^* = \frac{P}{P + R}. \tag{6}$$

The square is never negative and vanishes only at $K = K^*$. So K^*, the *Kalman gain*, is the unique best gain, and the smallest variance is

$$\mathrm{optVar}(P, R) = \frac{PR}{P + R}, \qquad \text{equivalently} \qquad \frac{1}{\mathrm{optVar}(P, R)} = \frac{1}{P} + \frac{1}{R}.$$

Precisions (one over variance) add. In particular optVar $\le P$ and optVar $\le R$: the blend beats both sources. Figure 10 gives a second view. For Gaussian errors, reading y slices the joint density of state and reading, and the slice is exactly the updated belief $\mathcal{N}(m, \mathrm{optVar})$. Figure 11 draws (6) as a valley whose floor is the Kalman gain.

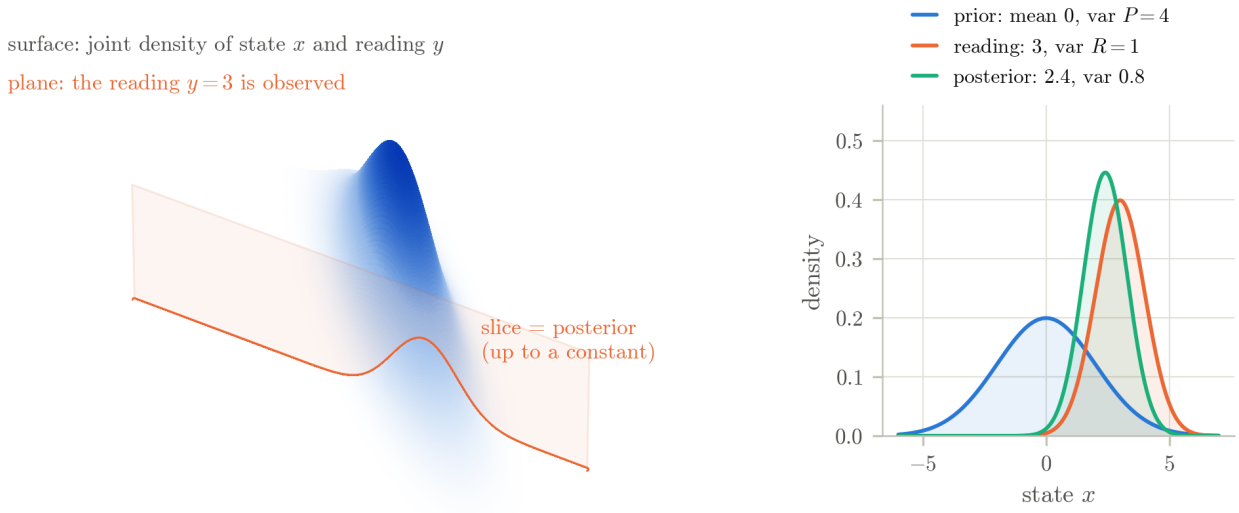


Figure 10. The update as a slice. Prior $\mathcal{N}(0, 4)$, reading noise variance 1, reading $y = 3$. Left: the joint density of state and reading; the orange curve is its slice at $y = 3$. Right: that slice, normalized, is the posterior, with mean $0.8 \cdot 3 = 2.4$ (gain $K^* = 0.8$) and variance $4 \cdot 1/(4 + 1) = 0.8$, narrower than the prior and than the reading.

Predict

Push the guess through one OU step (4): the next prediction is $a\,m$, and its error variance is $a^2\,\mathrm{optVar}(P, R) + Q$. Combining the two halves gives a recursion for the variance P_n just before the n-th reading, called the *Riccati recursion*:

$$P_{n+1} = a^2\,\frac{P_n R}{P_n + R} + Q. \tag{7}$$

This is the Lean definition `riccatiStep`, and `riccatiIter` iterates it from a start P_0. It does not depend on the readings, only on a, Q and R; for an OU mode started at equilibrium ($P_0 = \sigma^2$, $Q = \sigma^2(1 - a^2)$) it is `ouRiccati`. Figure 12 shows the filter at work, and Figure 13 shows that runs of (7) from different starts all meet at one fixed point P_∞.

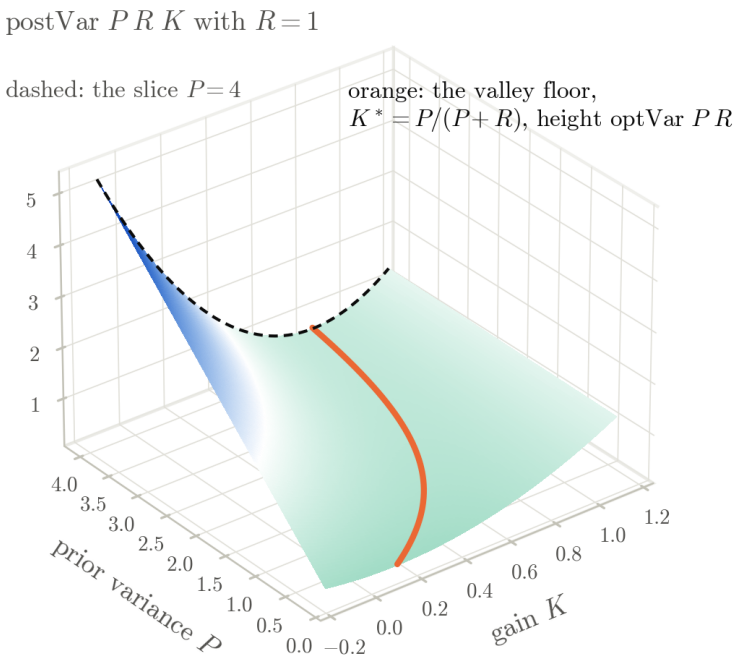


Figure 11. The update variance $\mathrm{postVar}(P, R, K)$ for $R = 1$, over the gain K and the prior variance P. For every P the minimum over K (orange) sits at $K^* = P/(P + R)$ with height $\mathrm{optVar}(P, R)$; the dashed parabola is the slice $P = 4$.

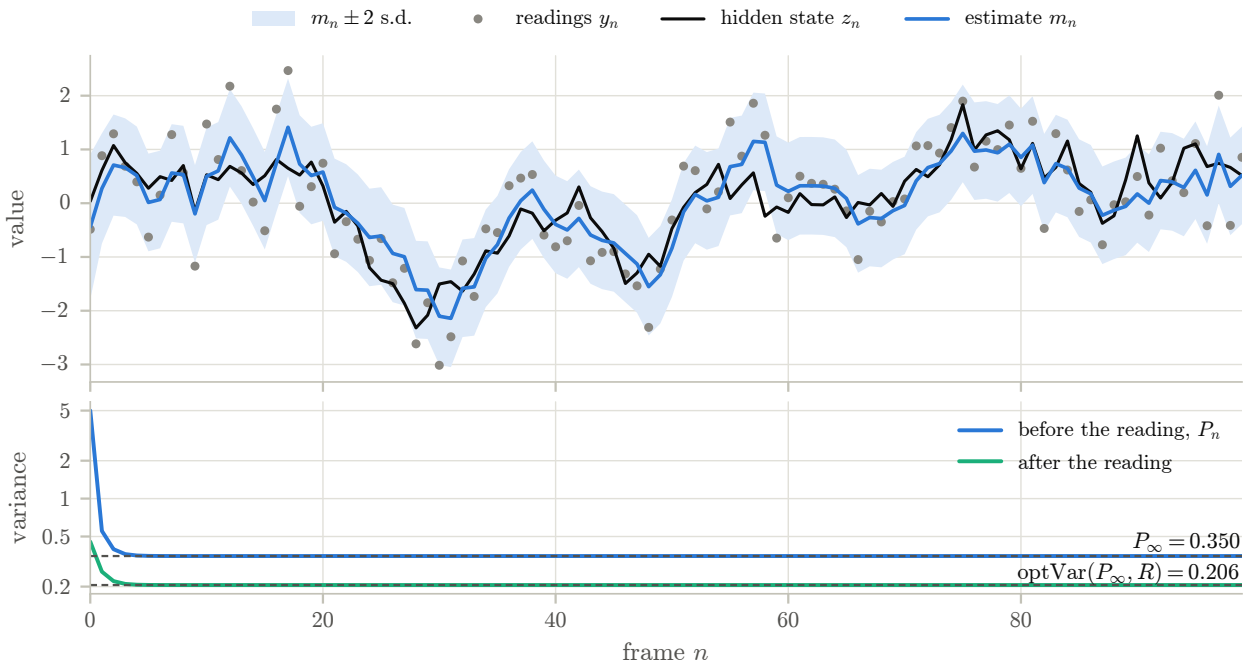


Figure 12. The filter on a simulated OU mode ($a = 0.905$, $Q = 0.181$, $R = 0.5$), started deliberately unsure ($P_0 = 5$). Top: after frame 20 the hidden state lies within two standard deviations of the estimate in 78 of 80 frames. Bottom: the variances before and after each reading settle within a few frames.

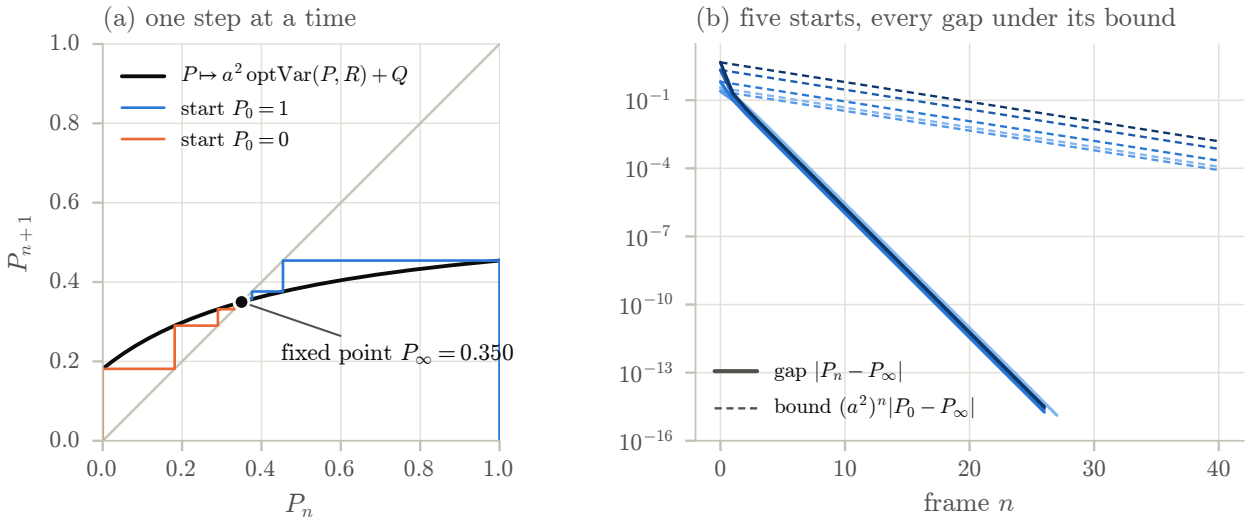


Figure 13. The recursion (7) for the mode of Figure 12. (a) Cobweb: each step goes up to the curve and across to the diagonal; both starts end at the fixed point. (b) The gap to the fixed point shrinks at least as fast as $(a^2)^n$, with $a^2 = 0.819$ (dashed). Near P_∞ it actually shrinks by about 0.283 per frame, and the largest ratio of gap to bound is 0.59.

> **Intuition:** *trust each source in proportion to its precision, then let the dynamics blur the result a little.*

9 Innovations: the filter's surprises

The *innovation* is the part of a reading the filter did not predict:

$$\nu_n = y_n - m_n^- = e_n + v_n.$$

If the model is right and the gain is the Kalman gain, then $\mathrm{Var}(\nu_n) = S_n = P_n + R$, and innovations at different frames are uncorrelated, or *white*. Both can be checked on data, channel by channel:

- **Size:** is the measured variance of the innovations, divided by S, close to 1?
- **Pattern:** is their autocorrelation close to 0 at every lag $k \geq 1$?

In the research run, a backbone torsion was deliberately hidden from the model, and one channel's innovations came out at about 4 times their expected level. The next section explains exactly what such a ratio means.

> **Intuition:** *a good model is surprised by the right amount, and never in a predictable way.*

10 Two ways the model can be wrong, and their fingerprints

An omitted coordinate (size). Suppose a channel's reading also contains a coordinate h that the model leaves out, independent of e and v, with $\mathrm{Var}(h) = \tau^2$. Then $\nu = e + v + h$ and

$$\frac{\mathrm{Var}(\nu)}{S} = \frac{S + \tau^2}{S} = 1 + \frac{\tau^2}{S}.$$

The observed 4x therefore means $\tau^2 \approx 3S$: the hidden torsion carried three times the variance the filter expected (Figure 14a).

A hidden switch (pattern). Suppose instead the filter ignores the basin label. For two basins the label is a Markov chain with transition matrix

$$M = \begin{pmatrix} 1-p & p \\ q & 1-q \end{pmatrix},$$

where p and q are the switching probabilities per frame (Figure 15). Its eigenvalues are 1 and $\lambda = 1 - p - q$, and $(p, -q)$ is an eigenvector for λ. Read as a function on the two states, $f(A) = p$ and $f(B) = -q$, this says $\mathbb{E}[f(s_{n+1}) \mid s_n] = \lambda f(s_n)$: the label's deviation from equilibrium shrinks by λ per frame. When switches are rare, λ is close to 1 and the label has a long memory. A filter that ignores the switch passes this memory into its innovations. The research model takes their lag covariances c_n to follow

$$c_{n+1} = \alpha\, c_n + \beta\, \lambda^{n+1},$$

where $\alpha = a(1 - K)$ is the filter's own decay per frame and β measures how strongly the switch leaks in. If $|\alpha| < |\lambda|$, then $c_{n+1}/c_n \to \lambda$: the tail of the autocorrelation decays at the switch's rate (Figure 14b).

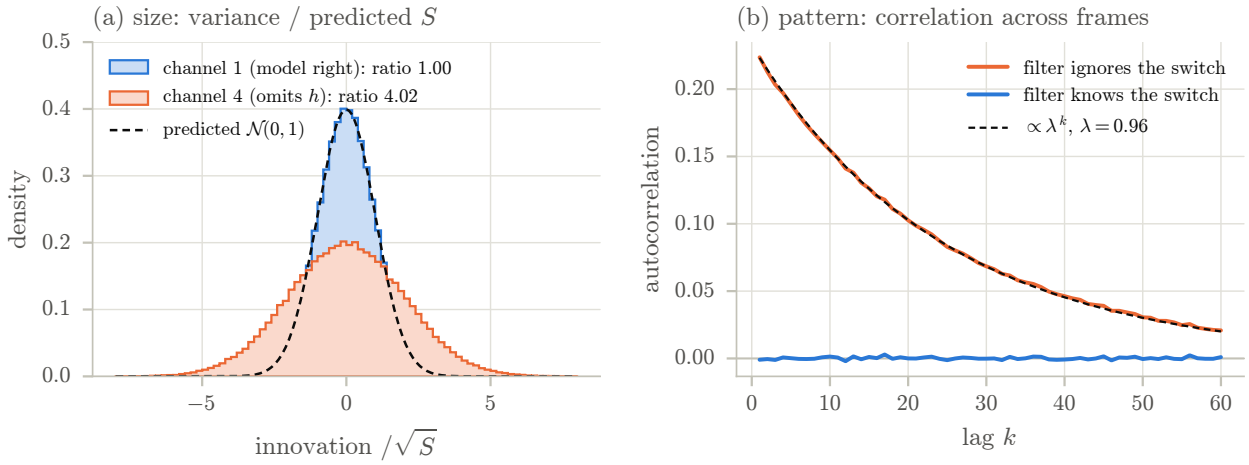


Figure 14. The two innovation tests on simulated data. (a) Size: with $\tau^2 = 3S$, channel 4 has variance ratio 4.02 against the theory value 4; a correctly modeled channel gives 1.00 (200,000 innovations each). (b) Pattern: a filter that ignores a two-state switch ($p = q = 0.02$, 1,000,000 frames) leaves correlations that decay like λ^k; a fit gives $\hat{\lambda} = 0.961$ against $\lambda = 0.96$. With the switch known, no lag exceeds 0.0029.

> **Intuition:** *surprises that are too large point to a missing coordinate; correlations that fade slowly point to a hidden switch.*

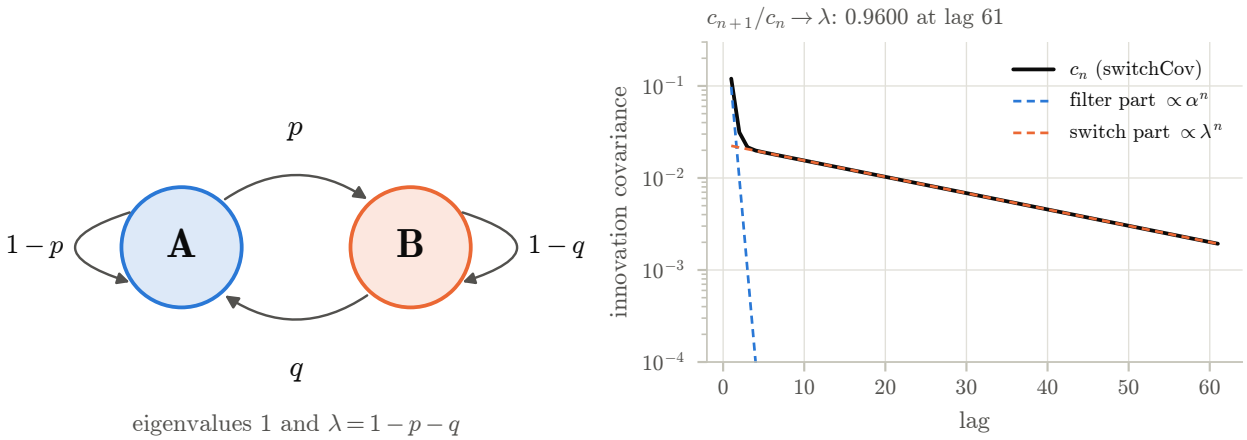


Figure 15. Left: the two-state switch with switching probabilities p and q per frame. Right: the lag covariances c_n (`switchCov`) for $\alpha = 0.101$, $\lambda = 0.96$: a fast filter part $\propto \alpha^n$ dies within a few lags, and the switch part $\propto \lambda^n$ sets the tail, so $c_{n+1}/c_n \to \lambda$.

PART II

What should be proved, and what is assumed

11 The verification agenda

Part I gives a model and a procedure. Before trusting a diagnosis made with them, one wants guarantees. Table 1 lists the questions, why each matters for an MD study, and the Lean result that answers it; Part III states each result exactly.

Question	Why it matters	Result
Is the Kalman update the best linear blend, and the only best one?	the filter's core step is right	Core
Does the blend beat both of its sources?	a reading never makes the estimate worse	Core
Can coordinate families be fused in any order, and does each help?	the answer cannot depend on bookkeeping	S
Can tracking an OU mode be worse than knowing only its spread?	the filter never loses track of a basin	O
Is (5) a theorem about random variables?	links the algebra to probability	A
Exactly how much does an omitted coordinate inflate the innovations?	turns the observed 4x into the size of what is missing	D, F5
Are white innovations a sure sign of the right gain?	then any pattern proves the gain or the model wrong	W
What fingerprint does a hidden switch leave?	identifies a missing basin from data	H0, H
Does the jump rule keep the intended populations?	the surrogate has the right equilibrium	M
Does the filter forget its start, settle, and stay calibrated?	justifies a burn-in and a single reference level	F

Table 1. What is worth proving about the model and the filter.

> **Intuition:** *prove the steps a diagnosis relies on, so that only the data can be wrong.*

12 Assumptions: what the theorems take as given

A theorem turns assumptions into conclusions; the data must still support the assumptions. Table 2 lists each one, the results that use it, and a way to check it on an MD run.

Four modeling simplifications deserve to be stated plainly.

- **Second moments only.** The recursions `ouRiccati`, `riccatiIter`, `innovCov` and `switchCov` are definitions of variances and covariances; Lean does not derive them from a stochastic process. Only A, D and F5 are proved from random variables.
- **Scalar.** One mode and one sensor at a time; no vectors or matrices of states.

Assumption	Used by	Empirical check
Each slow mode is linear-Gaussian (OU) with constant a and Q	O, F	each slow coordinate's autocorrelation decays like a^n; residuals look Gaussian
Sensor noise is independent of the state error and across channels	A, D, F5	cross-correlations of innovations between channels are near 0
Each channel's noise variance R is known and constant	F, W, D	channels without an omitted coordinate show ratios near 1 (Figure 14a)
The filter has reached its steady state	W, F5	burn-in discarded (Section 13); P_n flat afterwards
Jump rates follow the square-root rule at a fixed slow state	M	transition counts give $p_i k_{ij} \approx p_j k_{ji}$ (Section 4)
The switch is a constant-rate two-state chain	H0, H	dwell times in each basin are exponentially distributed

Table 2. The assumptions behind the results, and how to test each on data.

- **Linear-optimal.** K^* is the best *linear* update for any errors with the given variances. It is the best of all estimators only for Gaussian errors; heavy tails or several modes break that.
- **Approximate switch.** H uses the constant-rate two-state approximation of the switch, not the state-dependent rule (3); M holds at one fixed slow state.

Intuition: *the theorems are exact; whether they apply is a question for the data.*

13 What the theorems give an MD run

- **Burn-in.** With frame spacing Δt and relaxation time τ_c, $a = e^{-\Delta t/\tau_c}$, so by F the influence of the starting variance shrinks at least like $(a^2)^n = e^{-2n\Delta t/\tau_c}$. To shrink it below a fraction ε, discard
$$n \geq \frac{\tau_c}{2\Delta t} \ln \frac{1}{\varepsilon}$$
frames: 24 frames for $\varepsilon = 10^{-2}$ and 35 for $\varepsilon = 10^{-3}$ when $\tau_c = 10\,\Delta t$ (Figure 16a).
- **Reference level.** After burn-in, each channel's innovations should have variance $S_\infty = P_\infty + R$, where P_∞ is `steadyVar`.
- **Detectability.** An omitted coordinate gives exactly $1 + \tau^2/S_\infty$ (F5). With N white innovations the measured ratio fluctuates by about $\sqrt{2/N}$ (standard sampling theory, not proved in Lean), so an excess above $3\sqrt{2/N}$ stands out: 0.13 for $N = 1{,}000$ and 0.013 for $N = 100{,}000$ (Figure 16b).

Intuition: *discard the burn-in, then judge each channel against S_∞, above the noise floor.*

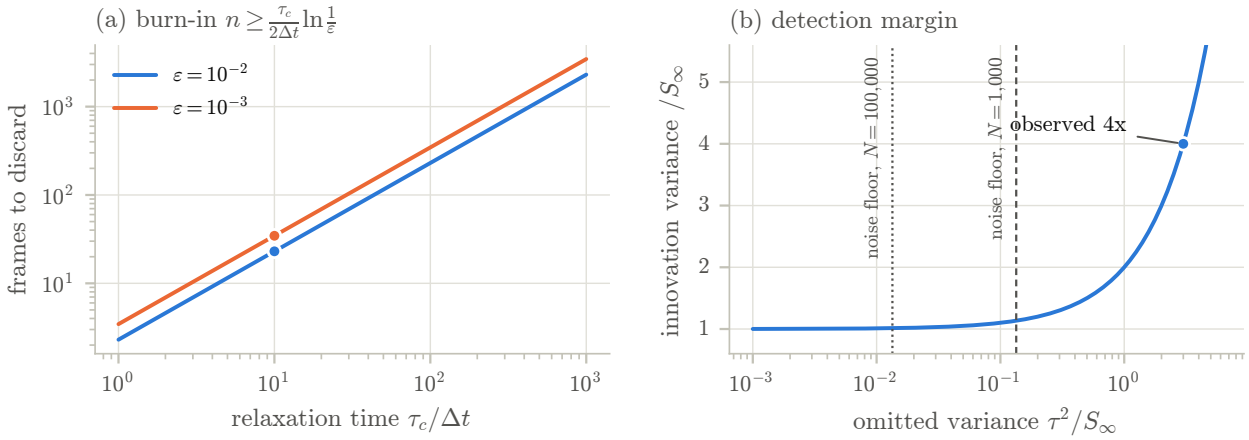


Figure 16. (a) Frames to discard before the start is forgotten, against the relaxation time. (b) The exact margin $1 + \tau^2/S_\infty$ of F5 against two sampling noise floors; the observed 4x is far above both.

PART III

The model in Lean

14 Lean in five minutes

- **What Lean does.** Lean is a programming language and a proof assistant. You write definitions, statements and proofs. A small trusted core, the *kernel*, checks every step of every proof; a proof is either accepted or rejected. *Mathlib* is Lean's mathematics library: real numbers, limits, matrices, probability.
- **What Lean does not do.** It checks that a proof proves the statement, not that the statement says what you meant. So the definitions and the final statements are read and approved by a person; the rest is left to the kernel.
- **Reading a statement.** `theorem name (P R : ℝ) (hP : 0 ≤ P) (hR : 0 < R) : C` means: for all real P and R with $0 \le P$ and $0 < R$, the claim C holds. The hypotheses have names (`hP`, `hR`) so that the proof can use them. Functions are applied without brackets: `gain P R` is gain(P, R).
- **Division is total.** In Lean $x/0 = 0$, so `gain P R = P / (P + R)` is defined even when $P + R = 0$. That is why the theorems carry hypotheses such as $0 < R$.
- **Axioms.** Lean and Mathlib use three standard axioms: propositional extensionality (`propext`), the axiom of choice (`Classical.choice`) and quotient soundness (`Quot.sound`). The command `#print axioms` lists what a theorem depends on. An unfinished proof would show up there as `sorryAx`.

Table 3 translates the notation that appears below.

Lean	Mathematics
$\mathbb{R}$, $\mathbb{N}$	the real numbers; the natural numbers $0, 1, 2, \dots$
`a ^ 2`, `|x|`, `Real.sqrt x`	a^2, $\lvert x\rvert$, $\sqrt{x}$
`∀ x, A`; `A → B`; `A ∧ B`; `A ↔ B`	for all x, A; if A then B; A and B; A if and only if B
`| 0 => u0 | n + 1 => g (u n)`	the sequence u_0, $u_{n+1} = g(u_n)$
`Tendsto u atTop (𝓝 L)`	$u_n \to L$ as $n \to \infty$
`Fin n`; `Matrix (Fin n) (Fin n) ℝ`	the indices $\{0, \dots, n-1\}$; real $n \times n$ matrices
`!![a, b; c, d]`; `![u, v]`	the matrix with rows (a, b) and (c, d); the vector (u, v)
`M.mulVec v`; `Matrix.vecMul p L`	Mv; $p^{\mathsf{T}}L$
`∑ k ∈ Finset.univ.erase i, f k`	$\sum_{k \neq i} f(k)$
Ω, μ, `[IsProbabilityMeasure μ]`	a probability space
`e : Ω → ℝ`	a random variable e
`MemLp e 2 μ`	$\mathbb{E}[e^2] < \infty$, so e has a variance
`IndepFun e v μ`	e and v are independent
`variance e μ`; `c • e`	$\mathrm{Var}(e)$; the random variable $c\,e$

Table 3. Reading Lean notation.

Intuition: *Lean guarantees that each proof matches its statement, not that the statement matches your idea.*

15 The file and its trust boundary

All the Lean work is in one file, `Kalman.lean`, divided into Parts (Figure 17). Parts 1 and 3 hold the 3 core and 11 model definitions, and Part 6 the 17 final statements: these are what a person reads. Parts 2, 4, 4b, 4c and 5 hold 38 proofs, which Lean checks. Each Part 6 theorem is proved in one line by citing a lemma from a proof Part. Part 1 and the six proofs of Part 2 are the student's own work, copied unchanged from `Core.lean`.

The trust argument is short. Suppose that (1) the definitions of Parts 1 and 3 mean what Part I says, (2) each Part 6 statement says what Section 17 says it says, and (3) Lean accepts the file and `#print axioms` lists only the three standard axioms. Then every statement is true. Lean guarantees (3); Sections 16 and 17 are there to help you check (1) and (2).

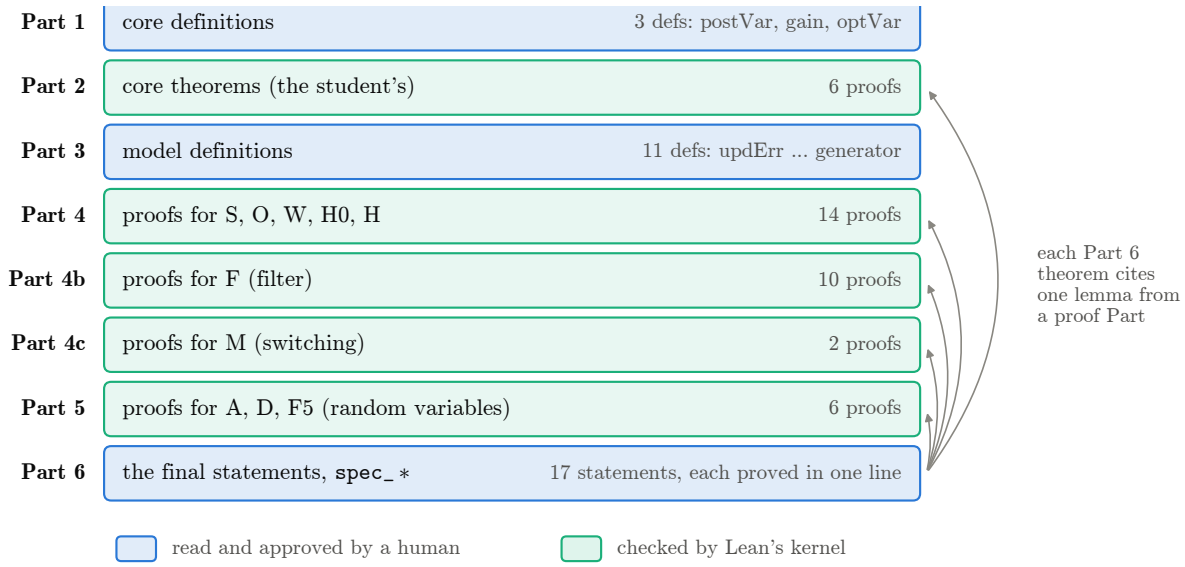


Figure 17. The Parts of `Kalman.lean`. Blue Parts are read and approved by a person, green Parts are checked by Lean's kernel; each final statement in Part 6 cites one lemma from a green Part.

Intuition: *read the blue Parts; the kernel checks the green ones.*

16 The definitions

Each excerpt below is copied verbatim from `Kalman.lean` by the figure script, so what you read is exactly what Lean checks. Two index conventions: `Fin n` counts from 0, and the sequences `innovCov` and `switchCov` start at lag 1, so index k holds lag $k+1$.

Part 1: the core update (the student's)

```
/-- Error variance after an update with gain K (slide 5). -/
def postVar (P R K : ℝ) : ℝ :=
  (1 - K)^2 * P + K^2 * R
```

The variance (5) of the update error $(1-K)e - Kv$, when $\operatorname{Var} e = P$ and $\operatorname{Var} v = R$.

```
/-- The Kalman gain (slide 6). -/
def gain (P R : ℝ) : ℝ := P / (P + R)
```

The Kalman gain $K^* = P/(P+R)$.

```
/-- The smallest possible variance, f(K*) (slide 6). -/
def optVar (P R : ℝ) : ℝ := P * R / (P + R)
```

The smallest update variance $PR/(P+R)$, reached at $K = K^*$.

Part 3: the model

```
/-- Error after an update with gain `K`, from prediction error `e` and sighting noise `v`. -/
def updErr {Ω : Type*} (K : ℝ) (e v : Ω → ℝ) : Ω → ℝ :=
  fun ω => (1 - K) * e ω - K * v ω
```

The update error as a random variable: a function on the sample space Ω, sending each outcome ω to $(1-K)\,e(\omega) - K\,v(\omega)$.

```
/-- Prediction step for one OU mode: decay `a`, stationary (equilibrium) variance `σ2`. -/
def ouPredict (a σ2 P : ℝ) : ℝ := a ^ 2 * P + σ2 * (1 - a ^ 2)
```

One OU prediction step for a variance: $P \mapsto a^2P + \sigma^2(1-a^2)$ (Section 6). Lean's `σ2` is the equilibrium variance σ^2.

```
/-- Prior variance of an OU mode under the optimal filter, started at equilibrium `σ2`. -/
def ouRiccati (a σ2 R : ℝ) : ℕ → ℝ
  | 0     => σ2
  | n + 1 => ouPredict a σ2 (optVar (ouRiccati a σ2 R n) R)
```

The prior variance of an OU mode under the optimal filter, started at equilibrium:

$$P_0 = \sigma^2, \qquad P_{n+1} = \mathrm{ouPredict}\big(\mathrm{optVar}(P_n, R)\big),$$

that is (7) with $Q = \sigma^2(1-a^2)$.

```
/-- Steady-state innovation autocovariance at lag `k + 1`.
Lag 1 is `a(1-K)P - aKR`; each further lag multiplies by the filter pole `a(1-K)`. -/
def innovCov (a P R K : ℝ) : ℕ → ℝ
  | 0     => a * (1 - K) * P - a * K * R
  | k + 1 => a * (1 - K) * innovCov a P R K k
```

Innovation autocovariances at steady state for a gain K: lag 1 is $a(1-K)P - aKR$, and each further lag multiplies by the filter's decay $a(1-K)$.

```
/-- Transition matrix of a two-state Markov state model, switching probabilities `p`, `q`. -/
def msm (p q : ℝ) : Matrix (Fin 2) (Fin 2) ℝ :=
  !![1 - p, p; q, 1 - q]
```

The two-state transition matrix $M = \begin{pmatrix} 1-p & p \\ q & 1-q \end{pmatrix}$ (a Markov state model, MSM).

```
/-- Innovation autocovariance at lag `n + 1` when an unmodeled switch with eigenvalue `lam`
leaks into the innovations with strength `β`; `α` is the filter pole, `c₁` the lag-1 value. -/
def switchCov (α β lam c₁ : ℝ) : ℕ → ℝ
```

```
  | 0     => c₁
  | n + 1 => α * switchCov α β lam c₁ n + β * lam ^ (n + 1)
```

Lag covariances when a hidden switch leaks in. Writing c_n for the value at index n (lag $n+1$): $c_0 = \gamma$, the lag-1 value that Lean calls `c₁`, and $c_{n+1} = \alpha\, c_n + \beta\, \lambda^{n+1}$ (Section 10).

```
/-- One step of the scalar filter variance: optimal update with noise `R`, then predict
with decay `a` and process noise `Q`. -/
def riccatiStep (a Q R P : ℝ) : ℝ := a ^ 2 * optVar P R + Q
```

One step of the variance recursion (7): update with noise R, then predict with decay a and process noise Q.

```
/-- Prior variance after `n` steps from initial variance `P₀`. -/
def riccatiIter (a Q R P₀ : ℝ) : ℕ → ℝ
  | 0     => P₀
  | n + 1 => riccatiStep a Q R (riccatiIter a Q R P₀ n)
```

The recursion run for n steps from P_0.

```
/-- Steady-state prior variance: the nonnegative root of `P² + (R(1-a²) - Q)P - QR = 0`. -/
def steadyVar (a Q R : ℝ) : ℝ :=
  ((Q - R * (1 - a ^ 2)) + Real.sqrt ((R * (1 - a ^ 2) - Q) ^ 2 + 4 * Q * R)) / 2
```

The candidate steady state P_∞. A fixed point $P = a^2PR/(P+R) + Q$ with $P + R > 0$ satisfies $P^2 + (R(1-a^2) - Q)\,P - QR = 0$; `steadyVar` is the root with the plus sign.

```
/-- Jump rate from a basin with population density `pᵢ` to one with `pⱼ`:
symmetric prefactor `κ` times `sqrt (pⱼ / pᵢ)`. -/
def jumpRate (κ pᵢ pⱼ : ℝ) : ℝ := κ * Real.sqrt (pⱼ / pᵢ)
```

The jump rule (3): $k_{ij} = \kappa_{ij}\sqrt{p_j/p_i}$.

```
/-- Generator of basin switching at a fixed slow state: off-diagonal entries are jump
rates, each diagonal entry is minus its row's total outflow. -/
def generator {n : ℕ} (κ : Fin n → Fin n → ℝ) (p : Fin n → ℝ) :
    Matrix (Fin n) (Fin n) ℝ :=
  fun i j => if i = j then -(∑ k ∈ Finset.univ.erase i, jumpRate (κ i k) (p i) (p k))
             else jumpRate (κ i j) (p i) (p j)
```

The generator L of Section 5: jump rates off the diagonal, and minus each row's total outflow on the diagonal.

17 The seventeen verified statements

Each statement is shown as it appears in Part 6, without its one-line proof. Below each one: what it says, and what it means for the model.

17.1 Core: the Kalman update

```
-- Core
/-- The Kalman gain gives the smallest update variance among all gains `K`. -/
theorem spec_gain_optimal (P R K : ℝ) (hP : 0 ≤ P) (hR : 0 < R) :
    postVar P R (gain P R) ≤ postVar P R K
```

> **In words:** for all real P, R, K with $P \geq 0$ and $R > 0$: $\mathrm{postVar}(P, R, K^*) \leq \mathrm{postVar}(P, R, K)$.
>
> **For the model:** no gain gives a smaller error variance after a reading than the Kalman gain.

```
/-- A gain that attains the smallest update variance is the Kalman gain. -/
theorem spec_gain_unique (P R K : ℝ) (hP : 0 ≤ P) (hR : 0 < R)
    (h : postVar P R K = postVar P R (gain P R)) : K = gain P R
```

In words: if moreover $\mathrm{postVar}(P, R, K) = \mathrm{postVar}(P, R, K^*)$, then $K = K^*$.

For the model: the best gain is unique, so "the" Kalman gain is well defined.

```
/-- The fused variance `P R / (P + R)` is at most both `P` and `R`. -/
theorem spec_optVar_le (P R : ℝ) (hP : 0 ≤ P) (hR : 0 < R) :
    optVar P R ≤ P ∧ optVar P R ≤ R
```

In words: $\mathrm{optVar}(P, R) \le P$ and $\mathrm{optVar}(P, R) \le R$.

For the model: after an update, the error is smaller than both the prediction's and the reading's.

17.2 S: sensor fusion

```
-- S: sensor fusion (each coordinate family is a sensor)
/-- Fusing sensors with noise `R₁` and `R₂` gives the same variance in either order. -/
theorem spec_fusion_comm (P R₁ R₂ : ℝ) (hP : 0 ≤ P) (h₁ : 0 < R₁) (h₂ : 0 < R₂) :
    optVar (optVar P R₁) R₂ = optVar (optVar P R₂) R₁
```

In words: for $P \ge 0$ and $R_1, R_2 > 0$: $\mathrm{optVar}(\mathrm{optVar}(P, R_1), R_2) = \mathrm{optVar}(\mathrm{optVar}(P, R_2), R_1)$.

For the model: fusing two coordinate families gives the same variance in either order.

```
/-- Fusing a second sensor never increases the variance. -/
theorem spec_fusion_mono (P R₁ R₂ : ℝ) (hP : 0 ≤ P) (h₁ : 0 < R₁) (h₂ : 0 < R₂) :
    optVar (optVar P R₁) R₂ ≤ optVar P R₁
```

In words: $\mathrm{optVar}(\mathrm{optVar}(P, R_1), R_2) \le \mathrm{optVar}(P, R_1)$.

For the model: each additional coordinate family can only help.

17.3 O: an OU mode stays within its equilibrium spread

```
-- O: an OU mode is never tracked worse than its equilibrium spread
/-- For `a² ≤ 1`, the OU prior variance stays in `[0, σ2]` at every step `n`. -/
theorem spec_ou_bound (a σ2 R : ℝ) (ha : a ^ 2 ≤ 1) (hσ : 0 ≤ σ2) (hR : 0 < R) (n : ℕ) :
    0 ≤ ouRiccati a σ2 R n ∧ ouRiccati a σ2 R n ≤ σ2
```

In words: if $a^2 \le 1$, $\sigma^2 \ge 0$ and $R > 0$, then every term P_n of `ouRiccati` satisfies $0 \le P_n \le \sigma^2$.

For the model: filtering an OU mode is never worse than knowing only its equilibrium spread.

17.4 A: the variance formula from random variables

```
-- A: the variance formula from random variables
/-- For independent `e`, `v` in `L²`: `Var((1 - K) e - K v) = postVar (Var e) (Var v) K`. -/
theorem spec_variance_update {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hind : IndepFun e v μ) (K : ℝ) :
    variance (updErr K e v) μ = postVar (variance e μ) (variance v μ) K
```

In words: on a probability space, if e and v are independent and have variances, then for every K: $\mathrm{Var}((1-K)e - Kv) = \mathrm{postVar}(\mathrm{Var}\, e, \mathrm{Var}\, v, K)$.

For the model: formula (5) is a theorem about random variables, not an assumption.

```
/-- If `Var v > 0`, the gain `Var e / (Var e + Var v)` minimizes the update error's variance. -/
theorem spec_variance_update_optimal {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hind : IndepFun e v μ) (hR : 0 < variance v μ) (K : ℝ) :
    variance (updErr (gain (variance e μ) (variance v μ)) e v) μ
      ≤ variance (updErr K e v) μ
```

In words: if moreover $\mathrm{Var}\, v > 0$, the gain $\mathrm{Var}\, e/(\mathrm{Var}\, e + \mathrm{Var}\, v)$ gives the update error with the smallest variance.

For the model: the Kalman gain is optimal for the actual random errors.

17.5 D: an omitted coordinate

```
-- D: an omitted coordinate inflates the innovation variance (the 4x diagnostic)
/-- With `e ⊥ v` and `e + v ⊥ h`: `Var(e + v + h) = (Var e + Var v) + Var h`. -/
theorem spec_innov_excess {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v h : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hh : MemLp h 2 μ) (hev : IndepFun e v μ) (hind : IndepFun (e + v) h μ) :
    variance (e + v + h) μ = (variance e μ + variance v μ) + variance h μ
```

In words: if e, v, h have variances, e is independent of v, and $e + v$ is independent of h: $\mathrm{Var}(e + v + h) = (\mathrm{Var}\, e + \mathrm{Var}\, v) + \mathrm{Var}\, h$.

For the model: an omitted coordinate adds its whole variance to the innovations.

```
/-- If also `S = Var e + Var v > 0` and `Var h > 0`, then `Var(e + v + h) / S > 1`. -/
theorem spec_innov_excess_ratio {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v h : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hh : MemLp h 2 μ) (hev : IndepFun e v μ) (hind : IndepFun (e + v) h μ)
    (hS : 0 < variance e μ + variance v μ) (hτ : 0 < variance h μ) :
    1 < variance (e + v + h) μ / (variance e μ + variance v μ)
```

In words: if also $S = \mathrm{Var}\, e + \mathrm{Var}\, v > 0$ and $\mathrm{Var}\, h > 0$: $\mathrm{Var}(e + v + h)/S > 1$.

For the model: any omitted coordinate with nonzero variance pushes the innovation ratio above 1.

17.6 W: white innovations

```
-- W: white innovations exactly at the Kalman gain
/-- For `a ≠ 0`: every innovation autocovariance vanishes iff `K = P / (P + R)`. -/
theorem spec_white_iff (a P R K : ℝ) (ha : a ≠ 0) (hP : 0 ≤ P) (hR : 0 < R) :
    (∀ k, innovCov a P R K k = 0) ↔ K = gain P R
```

In words: for $a \neq 0$, $P \geq 0$ and $R > 0$: every lag of `innovCov` is 0 if and only if $K = P/(P+R)$.

For the model: white innovations occur exactly at the Kalman gain, so any correlation means a wrong gain or a wrong model.

17.7 H0 and H: the hidden switch

```
-- H0, H (stretch): the hidden switch's fingerprint
/-- `(p, -q)` is an eigenvector of the MSM transition matrix with eigenvalue `1 - p - q`. -/
theorem spec_msm_eigen (p q : ℝ) :
    (msm p q).mulVec ![p, -q] = (1 - p - q) • ![p, -q]
```

In words: for all real p, q: $M\,(p,-q)^{\mathsf{T}} = (1-p-q)\,(p,-q)^{\mathsf{T}}$.

For the model: a deviation from the switch's equilibrium shrinks by $\lambda = 1-p-q$ per frame.

```
/-- If `β ≠ 0` and `|α| < |lam|`, the ratio of successive lags tends to `lam`. -/
theorem spec_switch_ratio (α β lam c₁ : ℝ) (hβ : β ≠ 0) (hlt : |α| < |lam|) :
    Tendsto (fun n => switchCov α β lam c₁ (n + 1) / switchCov α β lam c₁ n)
      atTop (𝓝 lam)
```

In words: if $\beta \neq 0$ and $|\alpha| < |\lambda|$, the lag covariances of `switchCov` satisfy $c_{n+1}/c_n \to \lambda$ as $n \to \infty$.

For the model: the slow tail of the innovations' autocovariance reveals the switch's eigenvalue.

17.8 M: the switching keeps its equilibrium

```
-- M1: the square-root rule satisfies detailed balance
/-- For positive populations, `pᵢ · rate(i → j) = pⱼ · rate(j → i)`. -/
theorem spec_detailed_balance (κ pᵢ pⱼ : ℝ) (hi : 0 < pᵢ) (hj : 0 < pⱼ) :
    pᵢ * jumpRate κ pᵢ pⱼ = pⱼ * jumpRate κ pⱼ pᵢ
```

In words: for $p_i, p_j > 0$ and every κ: $p_i \cdot \mathrm{jumpRate}(\kappa, p_i, p_j) = p_j \cdot \mathrm{jumpRate}(\kappa, p_j, p_i)$.

For the model: the square-root rule balances the flow between every pair of basins.

```
-- M2: the target populations are stationary for the switching generator
/-- With symmetric `κ` and positive `p`, the populations satisfy `pᵀ L = 0`. -/
theorem spec_populations_stationary {n : ℕ} (κ : Fin n → Fin n → ℝ) (p : Fin n → ℝ)
    (hκ : ∀ i j, κ i j = κ j i) (hp : ∀ i, 0 < p i) :
    Matrix.vecMul p (generator κ p) = 0
```

In words: if $\kappa_{ij} = \kappa_{ji}$ for all i, j and every $p_i > 0$, then $p^{\mathsf{T}} L = 0$ for $L = \mathrm{generator}(\kappa, p)$.

For the model: started at the target populations, the switching stays there: the surrogate keeps its equilibrium.

17.9 F: the filter forgets its start, converges, and is calibrated

```
-- F: the filter forgets its start, converges, and is calibrated at steady state
/-- For `a² < 1`: gaps shrink like `(a²)ⁿ`; `steadyVar` is the fixed point, limit, and white. -/
theorem spec_filter_guarantee (a Q R : ℝ) (ha : a ^ 2 < 1) (ha0 : a ≠ 0)
    (hQ : 0 ≤ Q) (hR : 0 < R) :
    (∀ P₀ P₀' : ℝ, 0 ≤ P₀ → 0 ≤ P₀' → ∀ n,
      |riccatiIter a Q R P₀ n - riccatiIter a Q R P₀' n| ≤ (a ^ 2) ^ n * |P₀ - P₀'|) ∧
    (0 ≤ steadyVar a Q R ∧ riccatiStep a Q R (steadyVar a Q R) = steadyVar a Q R) ∧
    (∀ P₀ : ℝ, 0 ≤ P₀ → Tendsto (riccatiIter a Q R P₀) atTop (𝓝 (steadyVar a Q R))) ∧
    (∀ k, innovCov a (steadyVar a Q R) R (gain (steadyVar a Q R) R) k = 0)
```

In words: if $a^2 < 1$, $a \neq 0$, $Q \geq 0$ and $R > 0$, then: (1) for all starts $P_0, P_0' \geq 0$ and all n, the two runs of (7) satisfy $|P_n - P_n'| \leq (a^2)^n |P_0 - P_0'|$; (2) $P_\infty = \mathrm{steadyVar}(a, Q, R)$ is nonnegative

and a fixed point; (3) every run with $P_0 \geq 0$ tends to P_∞; (4) at P_∞ with the Kalman gain, every innovation autocovariance is 0.

For the model: the filter forgets how it was started, settles to one steady state, and is calibrated there.

17.10 F5: the exact detection margin

```
-- F5: detection margin (exact form of the innovation-variance ratio)
/-- If `S = Var e + Var v > 0`, then `Var(e + v + h) / S = 1 + Var h / S`. -/
theorem spec_detection_margin {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v h : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hh : MemLp h 2 μ) (hev : IndepFun e v μ) (hind : IndepFun (e + v) h μ)
    (hS : 0 < variance e μ + variance v μ) :
    variance (e + v + h) μ / (variance e μ + variance v μ)
      = 1 + variance h μ / (variance e μ + variance v μ)
```

In words: under the hypotheses of D with $S = \operatorname{Var} e + \operatorname{Var} v > 0$: $\operatorname{Var}(e + v + h)/S = 1 + \operatorname{Var}(h)/S$.

For the model: the innovation ratio measures the omitted variance exactly: 4x means $\operatorname{Var} h = 3S$.

18 The verification record

The file is built with `lake build LoVe.Final_Project.Kalman` on Lean and Mathlib v4.33.0. The build ends with "Build completed successfully", with no errors, no warnings and no `sorry`. A search of the file for `sorry`, `admit` and `native_decide` finds nothing, and the file declares no axioms. The `#print axioms` line for each of the 17 statements reports (with the file-and-line prefixes removed):

```
'Kalman.spec_gain_optimal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_gain_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_optVar_le' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_fusion_comm' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_fusion_mono' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_ou_bound' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_variance_update' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_variance_update_optimal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_innov_excess' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_innov_excess_ratio' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_white_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_msm_eigen' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_switch_ratio' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_detailed_balance' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_populations_stationary' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_filter_guarantee' depends on axioms: [propext, Classical.choice, Quot.sound]
'Kalman.spec_detection_margin' depends on axioms: [propext, Classical.choice, Quot.sound]
Build completed successfully (8706 jobs).
```

Finally, a script compared the definitions and statements against their sources: Part 1 matches the student's `Core.lean`, and Parts 3 and 6 match the definitions and statements supplied for the project, character for character.

Intuition: *a clean build with only the three standard axioms means every statement above is proved.*

A How each result is proved

Each proof below is a short paper argument, followed by the chain of Lean lemmas that carries it out. All the lemmas are in `Kalman.lean`; the Mathlib facts they use are listed in Appendix B.

A.1 Core: complete the square

The key identity is (6). In Lean, `complete_square` unfolds the definitions, clears the denominator $P + R > 0$ (`denom_pos`) with `field_simp`, and finishes with `ring`; `postVar_gain` shows $\mathrm{postVar}(P, R, K^*) = \mathrm{optVar}(P, R)$. Then:

- **Optimal.** After rewriting with both identities, the claim is $\mathrm{optVar} \le (P + R)(K - K^*)^2 + \mathrm{optVar}$, a product of a positive number and a square.
- **Unique.** Equal variances force $(P + R)(K - K^*)^2 = 0$. Since $P + R \neq 0$, the square is 0, so $K = K^*$.
- **Beats both sources.** Clearing the denominator, $\mathrm{optVar} \le P$ becomes $PR \le P(P+R)$, true since $R \le P + R$ and $P \ge 0$; and $\mathrm{optVar} \le R$ becomes $PR \le R(P + R)$, which `nlinarith` proves.

The student's proof of optimality, verbatim:

```
/-- The Kalman gain minimizes the update variance. -/
theorem gain_optimal (P R K : ℝ) (hP : 0 ≤ P) (hR : 0 < R) :
    postVar P R (gain P R) ≤ postVar P R K := by
  have comp_sq := complete_square P R K hP hR
  have pvg := postVar_gain P R hP hR
  rw [comp_sq]
  rw [pvg]
  rw [le_add_iff_nonneg_left]
  have pos := denom_pos P R hP hR
  have square_plus := sq_nonneg (K - gain P R)
  have pos_or_zero := le_of_lt pos
  apply mul_nonneg pos_or_zero square_plus
```

A.2 S: sensor fusion

Both fusion orders simplify to $PR_1R_2/(PR_1 + PR_2 + R_1R_2)$ (`optVar_optVar`: `field_simp`, then `ring`), which is symmetric in R_1 and R_2. Monotonicity is `optVar_le` applied to the prior $\mathrm{optVar}(P, R_1) \ge 0$.

A.3 O: the OU bound

Induction on n, proving both bounds at once. If $0 \le x \le \sigma^2$, the update gives $0 \le \mathrm{optVar}(x, R) \le x \le \sigma^2$, and the prediction $a^2y + \sigma^2(1 - a^2)$ is a weighted average of $y \in [0, \sigma^2]$ and σ^2 with weights a^2 and $1 - a^2$ in $[0, 1]$, so it stays in $[0, \sigma^2]$ (`ouPredict_mem`, a `calc` chain with `gcongr`; then `ou_bound`).

A.4 A: the variance formula

Write the update error as $(1-K)\cdot e + (-K)\cdot v$ (`updErr_eq`). Scaled copies of independent variables are independent (`IndepFun.comp`), so the sum rule (`IndepFun.variance_add`) and the scaling rule (`variance_smul`) give $(1-K)^2 \operatorname{Var} e + K^2 \operatorname{Var} v$. Optimality then follows from `gain_optimal` with $P = \operatorname{Var} e \ge 0$ and $R = \operatorname{Var} v > 0$.

```
/-- `Var((1 - K) e - K v) = (1 - K)^2 Var e + K^2 Var v` for independent `e`, `v` in `L^2`. -/
theorem variance_update {Ω : Type*} [MeasurableSpace Ω] {μ : Measure Ω}
    [IsProbabilityMeasure μ] {e v : Ω → ℝ} (he : MemLp e 2 μ) (hv : MemLp v 2 μ)
    (hind : IndepFun e v μ) (K : ℝ) :
    variance (updErr K e v) μ = postVar (variance e μ) (variance v μ) K := by
  have hind' : IndepFun ((1 - K) • e) ((-K) • v) μ :=
    hind.comp (measurable_const_mul (1 - K)) (measurable_const_mul (-K))
  rw [updErr_eq, hind'.variance_add (he.const_smul (1 - K)) (hv.const_smul (-K)),
    variance_smul, variance_smul, postVar]
  ring
```

A.5 D and F5: the omitted coordinate

Apply the sum rule to $(e+v)+h$, then to $e+v$ (`innov_excess`). The ratio exceeds 1 because $S < S + \operatorname{Var} h$ and $S > 0$ (`one_lt_div`). For the exact margin, rewrite the numerator with `innov_excess`; then $(S+\tau^2)/S = 1 + \tau^2/S$ because $S \neq 0$ (`add_div`, `div_self`).

A.6 W: white innovations

By induction, lag $k+1$ is $(a(1-K))^k$ times lag 1 (`innovCov_closed`), and lag 1 equals $a\,(P+R)\,(K^* - K)$ (`innovCov_zero`). If every lag vanishes, lag 1 does, so $K = K^*$ because $a \neq 0$ and $P + R > 0$. Conversely, at $K = K^*$ lag 1 is 0, hence every lag is 0.

```
/-- All lags vanish iff `K` is the Kalman gain. -/
theorem white_iff (a P R K : ℝ) (ha : a ≠ 0) (hP : 0 ≤ P) (hR : 0 < R) :
    (∀ k, innovCov a P R K k = 0) ↔ K = gain P R := by
  have hne : a * (P + R) ≠ 0 := mul_ne_zero ha (denom_pos P R hP hR).ne'
  constructor
  · intro hwhite
    have hlag1 : a * (P + R) * (gain P R - K) = 0 := by
      rw [← innovCov_zero a P R K hP hR]
      exact hwhite 0
    exact (sub_eq_zero.1 ((mul_eq_zero.1 hlag1).resolve_left hne)).symm
  · rintro rfl k
    rw [innovCov_closed, innovCov_zero a P R _ hP hR, sub_self, mul_zero, mul_zero]
```

A.7 H0: the switch's eigenvector

Compute both entries of $M\,(p,-q)^{\mathsf{T}}$: $(1-p)p - pq = p(1-p-q)$ and $qp - (1-q)q = -q(1-p-q)$. In Lean: split into the two entries (`fin_cases`), unfold with `simp`, finish with `ring`.

A.8 H: the switch's decay rate

From $|\alpha| < |\lambda|$ we get $\lambda \neq 0$ and $\lambda \neq \alpha$ (`lam_ne`). Induction gives the closed form $c_n = (\gamma - B)\,\alpha^n + B\,\lambda^n$ with $B = \beta\lambda/(\lambda - \alpha)$ and $\gamma = c_0$ (`switchCov_closed`). Dividing by λ^n, $c_n/\lambda^n = (\gamma - B)(\alpha/\lambda)^n + B \to B$ since $|\alpha/\lambda| < 1$ (`switchCov_div_tendsto`). Finally $c_{n+1}/c_n = \lambda \cdot \dfrac{c_{n+1}/\lambda^{n+1}}{c_n/\lambda^n} \to \lambda \cdot \dfrac{B}{B} = \lambda$, using $B \neq 0$, which holds because $\beta \neq 0$ (`switch_ratio`).

A.9 M: detailed balance and stationary populations

M1. Split $\sqrt{p_j/p_i} = \sqrt{p_j}/\sqrt{p_i}$ and use $p_i/\sqrt{p_i} = \sqrt{p_i}$; both sides become $\kappa\sqrt{p_i}\sqrt{p_j}$:

```
/-- Detailed balance of the square-root rule: `p_i · rate(i → j) = p_j · rate(j → i)`. -/
theorem detailed_balance (κ pᵢ pⱼ : ℝ) (hi : 0 < pᵢ) (hj : 0 < pⱼ) :
    pᵢ * jumpRate κ pᵢ pⱼ = pⱼ * jumpRate κ pⱼ pᵢ := by
  unfold jumpRate
```

```
  rw [Real.sqrt_div hj.le, Real.sqrt_div hi.le]
  calc pᵢ * (κ * (√pⱼ / √pᵢ)) = κ * (pᵢ / √pᵢ) * √pⱼ := by ring
    _ = κ * (pⱼ / √pⱼ) * √pᵢ := by rw [Real.div_sqrt, Real.div_sqrt]; ring
    _ = pⱼ * (κ * (√pᵢ / √pⱼ)) := by ring
```

M2. Entry j of $p^{\mathsf{T}}L$ is $p_jL_{jj} + \sum_{i\neq j} p_iL_{ij}$. By M1 and $\kappa_{ij} = \kappa_{ji}$, each term p_ik_{ij} equals p_jk_{ji}, so the sum is p_j times the total outflow of j, which p_jL_{jj} cancels:

```
/-- With symmetric prefactors the target populations are stationary: `pᵀ L = 0`. -/
theorem populations_stationary {n : ℕ} (κ : Fin n → Fin n → ℝ) (p : Fin n → ℝ)
    (hκ : ∀ i j, κ i j = κ j i) (hp : ∀ i, 0 < p i) :
    Matrix.vecMul p (generator κ p) = 0 := by
  ext j
  have hflow : ∀ i ∈ Finset.univ.erase j,
      p i * generator κ p i j = p j * jumpRate (κ j i) (p j) (p i) := by
    intro i hi
    simp only [generator, if_neg (Finset.ne_of_mem_erase hi), hκ i j]
    exact detailed_balance _ _ _ (hp i) (hp j)
  simp only [Matrix.vecMul, dotProduct, Pi.zero_apply]
  rw [← Finset.add_sum_erase _ _ (Finset.mem_univ j), Finset.sum_congr rfl hflow,
    ← Finset.mul_sum]
  dsimp only [generator]
  rw [if_pos rfl]
  ring
```

A.10 F: the filter guarantee

1. **Two runs forget their difference.** For $x, y \geq 0$,

$$\mathrm{optVar}(x, R) - \mathrm{optVar}(y, R) = \frac{R^2}{(x+R)(y+R)}\,(x - y),$$

   and the factor lies in $[0, 1]$, so $|\mathrm{optVar}(x, R) - \mathrm{optVar}(y, R)| \leq |x - y|$ (`optVar_lipschitz`). Hence one step of (7) shrinks gaps by a^2 (`riccatiStep_lipschitz`). The iterates stay ≥ 0 because $Q \geq 0$ (`riccatiIter_nonneg`), and induction gives the $(a^2)^n$ bound (`riccati_contract`).

2. **A steady state.** With $D = (R(1-a^2) - Q)^2 + 4QR \geq (Q - R(1-a^2))^2$ we have $\sqrt{D} \geq |Q - R(1-a^2)|$, so $P_\infty \geq 0$ (`steadyVar_nonneg`). Squaring the root shows that P_∞ solves the quadratic, which after clearing $P_\infty + R > 0$ is the fixed-point equation (`steadyVar_fixed`).

3. **Convergence.** A run started at P_∞ stays there (`riccatiIter_steady`). Comparing any run with that one, $|P_n - P_\infty| \leq (a^2)^n |P_0 - P_\infty| \to 0$ (`riccati_tendsto`).

4. **Calibration.** The backward direction of W, applied at $P = P_\infty \geq 0$ (`steady_white`).

Part (1) holds for every a; the hypothesis $a^2 < 1$ is used only in (3), and $a \neq 0$ only in (4). The convergence proof, verbatim:

```
/-- From any nonnegative start the filter variance converges to the steady state. -/
theorem riccati_tendsto (a Q R : ℝ) (ha : a ^ 2 < 1) (hQ : 0 ≤ Q) (hR : 0 < R) (P₀ : ℝ)
    (h₀ : 0 ≤ P₀) : Tendsto (riccatiIter a Q R P₀) atTop (𝓝 (steadyVar a Q R)) := by
  have hP := steadyVar_nonneg a Q R hQ hR
  have hbound := (tendsto_pow_atTop_nhds_zero_of_lt_one (sq_nonneg a) ha).mul_const
    |P₀ - steadyVar a Q R|
  rw [zero_mul] at hbound
  rw [tendsto_iff_norm_sub_tendsto_zero]
  refine squeeze_zero (fun n => norm_nonneg _) (fun n => ?_) hbound
  have hgap := riccati_contract a Q R P₀ _ hQ hR h₀ hP n
  rw [riccatiIter_steady a Q R hQ hR n] at hgap
```

```
rwa [Real.norm_eq_abs]
```

B Mathlib facts used

Lemma	Meaning
`mul_nonneg`, `sq_nonneg`	$0 \le a$, $0 \le b \Rightarrow 0 \le ab$; $0 \le a^2$
`le_add_iff_nonneg_left`	$a \le b + a \iff 0 \le b$
`add_eq_right`; `mul_eq_zero`	$a + b = b \iff a = 0$; $ab = 0 \iff a = 0 \vee b = 0$
`eq_zero_of_pow_eq_zero`; `sub_eq_zero`	$a^n = 0 \Rightarrow a = 0$; $a - b = 0 \iff a = b$
`mul_le_mul_of_nonneg_left`	$b \le c$, $0 \le a \Rightarrow ab \le ac$
`div_le_one`; `one_lt_div`	for $0 < b$: $a/b \le 1 \iff a \le b$; $1 < a/b \iff b < a$
`abs_mul`; `abs_of_nonneg`	$\lvert ab\rvert = \lvert a\rvert \, \lvert b\rvert$; $0 \le a \Rightarrow \lvert a\rvert = a$
`mul_le_of_le_one_left`	$0 \le b$, $a \le 1 \Rightarrow ab \le b$
`Real.abs_le_sqrt`; `Real.sq_sqrt`	$a^2 \le b \Rightarrow \lvert a\rvert \le \sqrt{b}$; $0 \le a \Rightarrow (\sqrt{a})^2 = a$
`Real.sqrt_div`; `Real.div_sqrt`	$0 \le x \Rightarrow \sqrt{x/y} = \sqrt{x}/\sqrt{y}$; $x/\sqrt{x} = \sqrt{x}$
`tendsto_pow_atTop_nhds_zero_of_lt_one`	$0 \le r < 1 \Rightarrow r^n \to 0$
`tendsto_pow_atTop_nhds_zero_of_abs_lt_one`	$\lvert r\rvert < 1 \Rightarrow r^n \to 0$
`squeeze_zero`	$0 \le u_n \le v_n$ and $v_n \to 0 \Rightarrow u_n \to 0$
`Finset.add_sum_erase`; `Finset.mul_sum`	split one term off a sum; move a constant factor into a sum
`IndepFun.comp`	functions of independent variables are independent
`IndepFun.variance_add`	$\operatorname{Var}(X + Y) = \operatorname{Var} X + \operatorname{Var} Y$ for independent X, Y with variances
`variance_smul`; `variance_nonneg`	$\operatorname{Var}(cX) = c^2 \operatorname{Var} X$; $0 \le \operatorname{Var} X$

C Symbols

Symbol	Meaning	In Lean
V, θ	energy landscape; noise strength (temperature)	–
p_i	population of basin i	`p i`, $\mathtt{p}_i$
κ_{ij}, k_{ij}	symmetric prefactor; jump rate $i \to j$	κ `i j`; `jumpRate`
L	generator of the basin switching	`generator` κ `p`
τ_c, Δt	relaxation time; frame spacing	–
a, Q	OU decay per frame $e^{-\Delta t/\tau_c}$; process noise	`a`, `Q`
σ^2	equilibrium variance of an OU mode	σ`2`
e, v, h	prediction error; sensor noise; omitted coordinate	`e`, `v`, `h`
P, R	prediction variance $\operatorname{Var} e$; sensor noise variance $\operatorname{Var} v$	`P`, `R`
K, K^*	a gain; the Kalman gain $P/(P+R)$	`K`, `gain P R`
P_n, P_∞	prior variance after n frames; its steady state	`riccatiIter`, `steadyVar`
ν, S	innovation; its predicted variance $P+R$	–
τ^2	variance of an omitted coordinate	`variance h` μ
p, q, λ	switching probabilities; switch eigenvalue $1-p-q$	`p`, `q`, `lam`
α, β	filter decay $a(1-K)$; leak strength of the switch	α, β

D Reproducing everything

- **The proofs.** From the repository root (the folder with `lakefile.toml`): `lake exe cache get`, then `lake build LoVe.Final_Project.Kalman`.
- **The figures and numbers.** Run `python make_writeup_figures.py` in this folder. It needs numpy and matplotlib, uses fixed seeds, takes about 5 to 15 minutes, and rewrites `figures/`, `generated/numbers.tex` and the verbatim Lean excerpts in `generated/lean/`.
- **This document.** Compile `kalman_writeup.tex` with pdfLaTeX, XeLaTeX, LuaLaTeX or Tectonic, twice so the contents settle.